Coding Agent Exercises: AI Report

FamPlan, team Banana Five-O, SC2006 Lab 4 §3.3.4. 31 October 2026.

Authors: POLOYAPOY RON JOSHUA ABES (POLO0002), LIM RUI XIAN EDWARD (ELIM095), LE VIET HOANG ANH (HOANGANH003), ATHENA LIM HUI RU (ALIM102), MUHAMMAD ILHAM BIN IRWAN (MUHA0313).

Assistant: Claude Code running Claude Opus 5.5, an in-IDE style coding agent that reads the repository, edits files and runs commands. This report was drafted with the same assistant from its session records, for the team to review.

Exercise 1 (§3.3.2): rebuild the control class from scratch

An agent rebuilt ScoringController (prototype/server/src/scoring/score.ts), the class we black-box tested, and it passed every test on its first attempt with no follow-up prompts.

How we prompted it and what it was given

We ran the agent in an isolated git worktree and told it to delete score.ts without opening it. The prompt said: "You will regenerate a control class that has been deleted, as if from scratch", and required one full attempt before running anything, then at most three fixes. To stop it recovering the code, it was barred from git history, the main checkout, and documents that quote the logic (Lab 3, Lab 4, the specs and the README).

It was given our three scoring test files, the surrounding code (scoring/config.ts, neighbourhoods/store.ts and service.ts, shared/src), design Diagram D, and the relevant SRS sections (FR-DEC-01 to 03, the business rules, the ScoringController description and the design-to-code table).

Outcome

Check	Result on attempt 1
scoring.blackbox.test.ts	43 of 43 passed
whitebox.test.ts	9 of 9 passed
scoring.test.ts	10 of 10 passed
Full server suite, run by us afterwards with the rebuilt class	255 of 255 passed
Type check (tsc)	No errors
Effort	24 tool calls, about 4 minutes, no iteration

The rebuilt class was not merged, and a copy is saved as Lab4/AI_Experiment_1_Rebuilt_score.ts.txt. It is 152 lines against 137 in ours and has the same behaviour under test, but a different structure. It keeps input validation in a private helper instead of exporting assertValidScoringInput(), adds a fields map to ScoringInputError, and writes some of its own messages.

Reflection

The manual suggests retrying with only the function bodies removed if the agent fails, and we never needed that fallback. The agent's own report explains why: the tests did most of the work. They fix the exports, the argument list, the validation rules and even the explanation wording, while the diagram and SRS added little. The design alone would not have been enough, since it never mentions ScoringInputError.

The result should be read with care. Passing our white-box tests shows matching behaviour, not matching structure, so our basis-path table for calculateScore() may not describe the rebuilt code. The agent also had to guess where tests were silent, such as the rounding tolerance and how to treat Infinity. A weaker test

suite would probably have led to more wrong guesses. To raise the chance of success on a less tested class, we would give it more worked examples of expected output.

The exercise also found real errors in our documents. Diagram D said normalise() returned an integer from 0 to 100, but the code and tests use a decimal from 0 to 1. SRS §9.5 listed three arguments for calculateScore() when the tests pass five. We corrected both in SRS v1.6.

Exercise 2 (§3.3.3): implement a feature we had not finished

We chose the option "implement a feature you didn't have time to complete by hand". Our teacher pointed out that a commute preference must say how the family travels, and that saving several commute locations did not change the ranking.

How we prompted it and what it was given

This was an extended conversation in one session. We shared a screenshot of the teacher's notes and asked what needed to change. The agent traced the code and found that estimates always assumed public transport and only the first saved destination reached scoring. We then asked it to complete the work. It had the whole repository, the SRS, the test suite, a local database and a browser to try the app.

Iteration

The first version needed several corrections, most of which the agent found itself by running things:

- A test it wrote hung, because its mocked state returned a new object on every render and caused an endless update loop.
- The type check caught places where the old single destination was still passed.
- Using the app in a browser showed the destination rows squashed in Explore's filter panel, although all tests passed.
- Address search failed on machines with no OneMap login, because OneMap now adds a token notice to its public results. The agent fixed this and added tests.

We kept the design decisions and the release steps: each destination counts equally through a simple average, and the production migration and deployment stay with the team member who owns the Vercel account.

Outcome

The feature was merged to main: 32 files changed, a database migration and scoring configuration famplan-score-v1.2, with 14 new test cases. All 255 server and 28 client tests passed. With a test family on a local copy of our data, setting Work to Car moved the top three to Kebun Bahru 98, Serangoon Central 96 and Jelebu 92. Adding Grandparents in Bedok by public transport moved Katong into the top three in place of Jelebu.

Reflection

The outcome met our expectations, and the agent was strongest at tracing a change across the database, API, client pages and tests. It was not reliable without checks, though: the layout problem passed every automated test and only showed up in the running app. We also had to review its database steps carefully, since a migration could have damaged data. It avoided this by using a separate local copy. Running the app and reviewing the result remained essential, and every presenter still needs to be able to explain this code.